@satspath/p2p

Resolve & serve **SatsPath signed payment profiles peer-to-peer** over the Holepunch stack (Hyperswarm / HyperDHT), with NAT hole-punching — no central server and no local pre-registration of the receiver.

A wallet that embeds this SDK automatically gets a **random node address** and can publish or resolve `user@domain` payment profiles directly between devices.

> Transport + verification only. This SDK moves **only public, signed payment profiles**. It never moves funds, signs Bitcoin transactions, or broadcasts. Trust comes from the secp256k1 **signature**, not from the transport.

How it works

```
receiver device                          sender device

satspath export rodrigo@x.dev            node = new SatsPathP2PNode()
   → rodrigo.json                        node.address  ← auto-generated, random
node.publish(rodrigo.json)               await node.resolve("rodrigo@x.dev")
   topic = sha256("…:rodrigo@x.dev")           join same topic, DHT lookup
   announce on HyperDHT                hole-punch + connect
   serve profile to peers               receive → VERIFY SIGNATURE → return
```

- **Address.** Each node has an auto-generated random Ed25519 keypair (Hyperswarm's Noise identity). `node.address` is its hex public key — *not* a Bitcoin key and controls no funds.
- **Discovery.** The DHT topic is `sha256("satspath:p2p:v1:" + canonicalAlias)`, so publisher and resolver find each other from the alias alone.
- **Trust.** A resolved profile is accepted only if `verifySignedProfile` passes (secp256k1 ECDSA over `sha256(serde_json(profile))`, matching the Rust core) and its alias matches the request. A tampered profile is rejected.

Install

```
npm install @satspath/p2p
```

Wallet integration

```
import { SatsPathP2PNode } from "@satspath/p2p";

const node = new SatsPathP2PNode();
console.log("my P2P address:", node.address); // auto-generated, random

//   Receiver: publish your signed profile (from `satspath export`)
import { readFileSync } from "node:fs";
const signed = JSON.parse(readFileSync("rodrigo.json", "utf8"));
await node.publish(signed); // announces on the DHT, serves to peers

//   Sender (another device): resolve by alias, get a VERIFIED profile
const profile = await node.resolve("rodrigo@satspath.dev", { timeoutMs: 20000 });
// profile.profile.methods → route/pay with your own wallet
```

To persist a node's address across restarts, pass a saved keypair:

```
import { generateNodeKeyPair } from "@satspath/p2p";
const keyPair = generateNodeKeyPair(); // store keyPair.secretKey securely (it is NOT a wallet key)
const node = new SatsPathP2PNode({ keyPair });
```

Try it on two computers

On the **receiver's** machine (in the SatsPath repo, build the CLI first):

```
satspath export rodrigo@satspath.dev > rodrigo.json
cd sdk/satspath-p2p && npm install
node examples/publish.mjs ../../rodrigo.json      # leave running
```

On the **sender's** machine (different computer / network):

```
cd sdk/satspath-p2p && npm install
node examples/resolve.mjs rodrigo@satspath.dev    # prints the verified profile
```

DHT announce + lookup typically takes a few seconds to ~30s on a cold start.

API

Export	Purpose
`new SatsPathP2PNode({ keyPair?, swarm? })`	A node with an auto-generated (or supplied) random address.
`node.address`	The node's hex public-key address.
`node.publish(signed)`	Announce + serve a signed profile on its alias topic. Rejects an unsigned/invalid profile.
`node.resolve(alias, { timeoutMs })`	Resolve over the DHT; returns a **signature-verified** profile or rejects on timeout.
`node.destroy()`	Tear down the swarm.
`generateNodeKeyPair()`	A fresh random node keypair.
`topicForAlias(alias)`	The 32-byte DHT topic for an alias.
`verifySignedProfile(signed)`	Verify a SatsPath signature natively in JS.

Tests

```
npm test
```

Unit tests cover topic determinism, address randomness, and **verifying a real Rust-signed profile fixture** (proving JS canonicalization matches the Rust core), plus tamper rejection. The live two-node DHT round-trip is exercised via the examples above (it needs network access to the DHT and is not a CI unit test).

Safety

- No funds moved, nothing signed or broadcast — public signed profiles only.
- The node keypair is a P2P identity, **not** a wallet seed/spending key.
- A profile is trusted only after its signature verifies; the DHT/transport is never trusted on its own.